WEEK 17.1 AI ASSISTED

Chenchu Raju

2303A52206

BATCH 43

TASK1:

PROMPT: “Generate a Python script using Pandas to clean a customer dataset by handling missing values (age, income, city), removing duplicates, standardizing text fields, and encoding categorical variables like gender and city. Ensure the output is a cleaned dataframe ready for analysis.”

CODE:

[1]
0s

```python
import pandas as pd

# Load dataset
data = pd.read_csv("/content/shopping_trends.csv")

# Display original data
print("Original Data:")
print(data.head())

# -------------------------------
# 1. Handle Missing Values
# -------------------------------

# Fill missing numerical values with median
if 'Age' in data.columns:
    data['Age'].fillna(data['Age'].median(), inplace=True)

if 'Income' in data.columns:
    data['Income'].fillna(data['Income'].median(), inplace=True)

# Fill missing categorical values with mode
if 'City' in data.columns:
    data['City'].fillna(data['City'].mode()[0], inplace=True)

# -------------------------------
# 2. Remove Duplicates
# -------------------------------
data.drop_duplicates(inplace=True)

# -------------------------------
# 3. Standardize Text Fields
# -------------------------------
if 'City' in data.columns:
    data['City'] = data['City'].str.lower().str.strip()
```

```python
if 'Gender' in data.columns:
    data['Gender'] = data['Gender'].str.lower().str.strip()

# -------------------------------
# 4. Encode Categorical Variables
# -------------------------------

# One-hot encoding for City
if 'City' in data.columns:
    data = pd.get_dummies(data, columns=['City'], drop_first=True)

# Label encoding for Gender
if 'Gender' in data.columns:
    data['Gender'] = data['Gender'].map({'male': 0, 'female': 1})

# -------------------------------
# Final Output
# -------------------------------
print("\nCleaned Data:")
print(data.head())

# Save cleaned dataset
data.to_csv("cleaned_shopping_trends.csv", index=False)
```

Output:

```
Cleaned Data:
   Customer ID  Age  Gender Item Purchased  Category  Purchase Amount (USD)  \
0            1   55       0        Blouse  Clothing                     53
1            2   19       0       Sweater  Clothing                     64
2            3   50       0         Jeans  Clothing                     73
3            4   21       0       Sandals  Footwear                     90
4            5   45       0        Blouse  Clothing                     49

        Location Size      Color  Season  Review Rating Subscription Status  \
0       Kentucky    L       Gray  Winter            3.1                 Yes
1          Maine    L     Maroon  Winter            3.1                 Yes
2  Massachusetts    S     Maroon  Spring            3.1                 Yes
3   Rhode Island    M     Maroon  Spring            3.5                 Yes
4         Oregon    M  Turquoise  Spring            2.7                 Yes

  Payment Method  Shipping Type Discount Applied Promo Code Used  \
0    Credit Card        Express              Yes             Yes
1  Bank Transfer        Express              Yes             Yes
2           Cash  Free Shipping              Yes             Yes
3         PayPal   Next Day Air              Yes             Yes
4           Cash  Free Shipping              Yes             Yes

   Previous Purchases Preferred Payment Method Frequency of Purchases
0                  14                    Venmo            Fortnightly
1                   2                     Cash            Fortnightly
2                  23              Credit Card                 Weekly
3                  49                   PayPal                 Weekly
4                  31                   PayPal               Annually
/tmp/ipykernel_3653/1941460342.py:16: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behav

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) ins


  data['Age'].fillna(data['Age'].median(), inplace=True)
```

Summary: This script cleans a customer dataset by handling missing values, removing duplicates, and standardizing text fields like city and gender. It also converts categorical data into numerical format using encoding techniques. The final cleaned dataset is saved and ready for further analysis or machine learning tasks.

Task 2:

Prompt: "Generate a Python script using Pandas to preprocess sales data by converting date columns to datetime, creating time-based features (month, year, day of week), normalizing the sales column, and detecting and handling outliers using the IQR method. Ensure the output dataset is ready for analysis."

Code:

[2]
✓ 0s

```python
import pandas as pd
import numpy as np

# Load dataset
data = pd.read_csv("/content/sales_data.csv")

# Display original data
print("Original Data:")
print(data.head())

# -------------------------------
# 1. Convert Date Column
# -------------------------------
if 'Date' in data.columns:
    data['Date'] = pd.to_datetime(data['Date'], errors='coerce')

# -------------------------------
# 2. Create Time-Based Features
# -------------------------------
if 'Date' in data.columns:
    data['Year'] = data['Date'].dt.year
    data['Month'] = data['Date'].dt.month
    data['DayOfWeek'] = data['Date'].dt.day_name()

# -------------------------------
# 3. Normalize Sales Column
# -------------------------------
if 'Sales' in data.columns:
    data['Sales_Normalized'] = (data['Sales'] - data['Sales'].min()) / \
                               (data['Sales'].max() - data['Sales'].min())

# -------------------------------
# 4. Detect & Handle Outliers (IQR)
# -------------------------------
if 'Sales' in data.columns:
```

```
[2]
0s
# -------------------------------
if 'Sales' in data.columns:
    Q1 = data['Sales'].quantile(0.25)
    Q3 = data['Sales'].quantile(0.75)
    IQR = Q3 - Q1

    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR

    median_value = data['Sales'].median()

    data['Sales'] = np.where(
        (data['Sales'] < lower_bound) | (data['Sales'] > upper_bound),
        median_value,
        data['Sales']
    )

# -------------------------------
# Final Output
# -------------------------------
print("\nProcessed Data:")
print(data.head())

# Save dataset
data.to_csv("processed_sales_data.csv", index=False)
```

Output:

```
  Product_Category  Unit_Cost  Unit_Price Customer_Type  Discount  \
0        Furniture     152.75      267.22     Returning      0.09
1        Furniture    3816.39     4209.44     Returning      0.11
2             Food     261.56      371.40     Returning      0.20
3         Clothing    4330.03     4467.75           New      0.02
4      Electronics     637.37      692.71           New      0.08

  Payment_Method Sales_Channel Region_and_Sales_Rep
0           Cash        Online            North-Bob
1           Cash        Retail             West-Bob
2  Bank Transfer        Retail          South-David
3    Credit Card        Retail            South-Bob
4    Credit Card        Online         East-Charlie

Processed Data:
   Product_ID   Sale_Date Sales_Rep Region  Sales_Amount  Quantity_Sold  \
0        1052  2023-02-03       Bob  North       5053.97             18
1        1093  2023-04-21       Bob   West       4384.02             17
2        1015  2023-09-21     David  South       4631.23             30
3        1072  2023-08-24       Bob  South       2167.94             39
4        1061  2023-03-24   Charlie   East       3750.20             13

  Product_Category  Unit_Cost  Unit_Price Customer_Type  Discount  \
0        Furniture     152.75      267.22     Returning      0.09
1        Furniture    3816.39     4209.44     Returning      0.11
2             Food     261.56      371.40     Returning      0.20
3         Clothing    4330.03     4467.75           New      0.02
4      Electronics     637.37      692.71           New      0.08

  Payment_Method Sales_Channel Region_and_Sales_Rep
0           Cash        Online            North-Bob
1           Cash        Retail             West-Bob
2  Bank Transfer        Retail          South-David
3    Credit Card        Retail            South-Bob
4    Credit Card        Online         East-Charlie
```

Summary: This script preprocesses sales data by converting date columns into datetime format and extracting useful time-based features like year, month, and day of the week. It normalizes the sales values for better analysis and applies the IQR method to detect and handle outliers. The final dataset is clean, structured, and ready for analytics or machine learning.

TASK 3:

Prompt: "Generate a Python script using Pandas and Scikit-learn to preprocess a healthcare dataset by handling missing values in blood pressure and cholesterol, encoding categorical variables, scaling numerical features, and splitting the dataset into training and testing sets for machine learning."

Code:

```
[3]
import pandas as pd
from sklearn.preprocessing import MinMaxScaler, LabelEncoder
from sklearn.model_selection import train_test_split

# Load dataset
data = pd.read_csv("/content/dirty_v3_path.csv")

# Display original data
print("Original Data:")
print(data.head())

# -------------------------------
# 1. Handle Missing Values
# -------------------------------
if 'blood_pressure' in data.columns:
    data['blood_pressure'].fillna(data['blood_pressure'].median(), inplace=True)

if 'cholesterol' in data.columns:
    data['cholesterol'].fillna(data['cholesterol'].median(), inplace=True)

# -------------------------------
# 2. Encode Categorical Variables
# -------------------------------
label_encoders = {}

for col in data.select_dtypes(include=['object']).columns:
    le = LabelEncoder()
    data[col] = le.fit_transform(data[col].astype(str))
    label_encoders[col] = le

# -------------------------------
# 3. Scale Numerical Features
# -------------------------------
scaler = MinMaxScaler()
```

```python
label_encoders = {}

for col in data.select_dtypes(include=['object']).columns:
    le = LabelEncoder()
    data[col] = le.fit_transform(data[col].astype(str))
    label_encoders[col] = le

# -------------------------------
# 3. Scale Numerical Features
# -------------------------------
scaler = MinMaxScaler()

num_cols = data.select_dtypes(include=['int64', 'float64']).columns
data[num_cols] = scaler.fit_transform(data[num_cols])

# -------------------------------
# 4. Train-Test Split
# -------------------------------
target_column = data.columns[-1]

X = data.drop(target_column, axis=1)
y = data[target_column]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# -------------------------------
# Final Output
# -------------------------------
print("\nPreprocessed Data:")
print(data.head())

print("\nTraining Set Shape:", X_train.shape)
print("Testing Set Shape:", X_test.shape)
```

Output:

```
Original Data:
    Age  Gender Medical Condition  Glucose  Blood Pressure    BMI  \
0  46.0    Male          Diabetes   137.04          135.27  28.90
1  22.0    Male           Healthy    71.58          113.27  26.29
2  50.0     NaN            Asthma    95.24             NaN  22.53
3  57.0     NaN           Obesity      NaN          130.53  38.47
4  66.0  Female      Hypertension    95.15          178.17  31.12

   Oxygen Saturation  LengthOfStay  Cholesterol  Triglycerides  HbA1c  \
0              96.04             6       231.88         210.56   7.61
1              97.54             2       165.57         129.41   4.91
2              90.31             2       214.94         165.35   5.60
3              96.60             5       197.71         182.13   6.92
4              94.90             4       259.53         115.85   5.98

   Smoking  Alcohol  Physical Activity  Diet Score  Family History  \
0        0        0              -0.20        3.54               0
1        0        0               8.12        5.90               0
2        0        0               5.01        4.65               1
3        0        0               3.16        3.37               0
4        0        1               3.56        3.40               0

   Stress Level  Sleep Hours random_notes   noise_col
0          5.07         6.05        lorem -137.057211
1          5.87         7.72        ipsum  -11.230610
2          3.09         4.82        ipsum   98.331195
3          3.01         5.33        lorem   44.187175
4          6.38         6.64        lorem   44.831426

Preprocessed Data:
        Age  Gender  Medical Condition   Glucose  Blood Pressure       BMI  \
0  0.455696     0.5           0.428571  0.391428        0.401144  0.431680
1  0.151899     0.5           0.571429  0.171904        0.256540  0.378609
2  0.506329     1.0           0.142857  0.251249             NaN  0.302155
3  0.594937     1.0           0.857143       NaN        0.369988  0.626271
4  0.708861     0.0           0.714286  0.250947        0.683121  0.476820
```

```
   Stress Level  Sleep Hours random_notes   noise_col
0          5.07         6.05        lorem -137.057211
1          5.87         7.72        ipsum  -11.230610
2          3.09         4.82        ipsum   98.331195
3          3.01         5.33        lorem   44.187175
4          6.38         6.64        lorem   44.831426

Preprocessed Data:
        Age  Gender  Medical Condition   Glucose  Blood Pressure       BMI  \
0  0.455696     0.5           0.428571  0.391428        0.401144  0.431680
1  0.151899     0.5           0.571429  0.171904        0.256540  0.378609
2  0.506329     1.0           0.142857  0.251249             NaN  0.302155
3  0.594937     1.0           0.857143       NaN        0.369988  0.626271
4  0.708861     0.0           0.714286  0.250947        0.683121  0.476820

   Oxygen Saturation  LengthOfStay  Cholesterol  Triglycerides     HbA1c  \
0           0.670348      0.277778     0.518390       0.524877  0.476872
1           0.705592      0.055556     0.265915       0.342102  0.179515
2           0.535714      0.055556     0.453891       0.423050  0.255507
3           0.683506      0.222222     0.388288       0.460844  0.400881
4           0.643562      0.166667     0.623667       0.311561  0.297357

   Smoking  Alcohol  Physical Activity  Diet Score  Family History  \
0      0.0      0.0           0.216283    0.383056             0.0
1      0.0      0.0           0.733375    0.553946             0.0
2      0.0      0.0           0.540087    0.463432             1.0
3      0.0      0.0           0.425109    0.370746             0.0
4      0.0      1.0           0.449969    0.372918             0.0

   Stress Level  Sleep Hours  random_notes  noise_col
0      0.419788     0.509132      1.000000   0.312605
1      0.464505     0.699772      0.666667   0.455579
2      0.309111     0.368721      0.666667   0.580072
3      0.304639     0.426941      1.000000   0.518549
4      0.493013     0.576484      1.000000   0.519281

Training Set Shape: (24000, 19)
Testing Set Shape: (6000, 19)
```

Summary:

This script prepares a healthcare dataset by filling missing values in key medical fields like blood pressure and cholesterol. It encodes categorical features, scales numerical data for consistency, and splits the dataset into training and testing sets. The result is a clean and structu;red dataset ready for machine learning models.

TASK 4:

PROMPT: “Generate a Python script using Pandas and Scikit-learn to preprocess an employee salary dataset by handling missing values, removing salary outliers, standardizing department names, and applying label encoding to job location and department columns.”

Code:

[7]
0s

```python
import pandas as pd
from sklearn.preprocessing import LabelEncoder

# Load dataset
data = pd.read_csv("/content/Department_Dataset.csv")

# Display original data
print("Original Data:")
print(data.head())

# -------------------------------
# 1. Handle Missing Values
# -------------------------------
num_cols = data.select_dtypes(include=['int64', 'float64']).columns
for col in num_cols:
    data[col].fillna(data[col].median(), inplace=True)

cat_cols = data.select_dtypes(include=['object']).columns
for col in cat_cols:
    data[col].fillna(data[col].mode()[0], inplace=True)

# -------------------------------
# 2. Remove Salary Outliers (IQR)
# -------------------------------
salary_col = 'salary' if 'salary' in data.columns else data.columns[-1]

# Check if the identified salary_col is numerical before proceeding with outlier removal
if pd.api.types.is_numeric_dtype(data[salary_col]):
    Q1 = data[salary_col].quantile(0.25)
    Q3 = data[salary_col].quantile(0.75)
    IQR = Q3 - Q1

    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR
```

```
        Q3 = data[salary_col].quantile(0.75)
        IQR = Q3 - Q1

        lower_bound = Q1 - 1.5 * IQR
        upper_bound = Q3 + 1.5 * IQR

        data = data[(data[salary_col] >= lower_bound) & (data[salary_col] <= upper_bound)]
    else:
        print(f"Skipping outlier removal for '{salary_col}' as it is not a numerical column.")

    # -------------------------------
    # 3. Standardize Department Names
    # -------------------------------
    if 'department' in data.columns:
        data['department'] = data['department'].str.lower().str.strip()

    # -------------------------------
    # 4. Label Encoding
    # -------------------------------
    le = LabelEncoder()

    for col in ['job_location', 'department']:
        if col in data.columns:
            data[col] = le.fit_transform(data[col])

    # -------------------------------
    # Final Output
    # -------------------------------
    print("\nProcessed Data:")
    print(data.head())

    # Save dataset
    data.to_csv("processed_employee_salary.csv", index=False)
```

```
Original Data:
   ID Dept_name   location travel_required
```

Output:

```
Original Data:
   ID Dept_name   location travel_required
0   1        HR       Pune             yes
1   2   Finance  Bangalore              no
2   3   Finance  Bangalore              no
3   4   Finance       Pune              no
4   5      Tech     Mumbai              no
Skipping outlier removal for 'travel_required' as it is not a numerical column.

Processed Data:
   ID Dept_name   location travel_required
0   1        HR       Pune             yes
1   2   Finance  Bangalore              no
2   3   Finance  Bangalore              no
3   4   Finance       Pune              no
4   5      Tech     Mumbai              no
/tmp/ipykernel_3653/312247396.py:16: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inp
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves a

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instea


  data[col].fillna(data[col].median(), inplace=True)
/tmp/ipykernel_3653/312247396.py:20: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inp
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves a

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instea


  data[col].fillna(data[col].mode()[0], inplace=True)
```

Summary:

This script preprocesses employee salary data by handling missing values and removing salary outliers using the IQR method. It standardizes department names for consistency and encodes categorical fields like job location and department. The final dataset is clean and structured for HR analytics and decision-making.